Day 4 – Authentication & Permissions

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Virtual Environment active**: Ensure your terminal shows `(.venv)` in the prompt.
   - **Django Server running**: Run `python manage.py runserver` to ensure the project boots up.
   - **HTTP Client Ready**: Have Postman, curl, or HTTPie open to make HTTP requests with custom headers.

2. **Prerequisites**:
   - You must have completed Day 3 successfully (DRF viewsets and routers configured, browsable API accessible).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain different API authentication schemes (Session, Token, JWT) and why stateless Token Auth fits REST APIs.
- Configure DRF Token Authentication globally.
- Create custom object-level permission classes in DRF to protect resources from unauthorized modifications.
- Build a user registration endpoint that hashes passwords securely and returns authentication tokens.
- Make authenticated API requests using custom request headers.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus
1. Hook & Big Picture	15 min	Compare API authentication strategies. Explain why stateless token-based auth is best for REST backends.
2. Key Concepts & Core Ideas	15 min	Distinguish authentication vs authorization. Explain DRF request lifecycle and permission checks.
3. Live Coding Walkthrough	45 min	Configure Token Auth, create custom permissions, write the register view, and test the endpoints.

4. Check for Understanding	10 min	Q&A on object-level permissions, password hashing, and token header formats.
5. Class Wrap-up & Checkpoint	5 min	Verify registration generates a token and unauthenticated writes are blocked.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

Right now, anyone in the world can run a POST request to add a game to our store. How do we secure our API so only authenticated publishers can edit games, and normal users can only view them?

- **Authentication Options**: Compare standard session authentication (cookies/browser-based) with token authentication (header-based) and JSON Web Tokens (JWT). For stateless web APIs, header-based Token Auth is simple, secure, and robust.
- **Q&A**:
  - **Question**: Why is token authentication the right choice for a stateless API?
    - *Answer*: Stateless APIs do not maintain session state on the server. Token authentication allows the client to send a secure token in each HTTP header, which the server validates on every request without keeping active session storage.

2. Key Concepts & Core Ideas (15 min)

Introduce these essential terms:

- **Authentication vs Authorization**:
  - *Authentication* answers: *"Who are you?"* (handled by TokenAuthentication).
  - *Authorization* answers: *"What are you allowed to do?"* (handled by PermissionClasses).
- **`IsAuthenticatedOrReadOnly`**: A global permission class that allows unauthenticated read operations (`GET`, `HEAD`, `OPTIONS`) but restricts write operations (`POST`, `PATCH`, `DELETE`) to authenticated users.
- **`BasePermission`**: The base class for custom permission logic. Explain the two methods we can override:
  - `has_permission(request, view)`: Evaluated at the start of the request (request-level check).
  - `has_object_permission(request, view, obj)`: Evaluated only during detail actions (`retrieve`, `update`, `destroy`) to check access to a specific database record (object-level check).
- **`SAFE_METHODS`**: Read-only HTTP methods: `GET`, `HEAD`, `OPTIONS`.
- **Password Hashing**: Storing plaintext passwords is a major security vulnerability. Django hashes passwords using PBKDF2.

3. Live Coding Walkthrough (45 min)

Step 3.1: Enable Token Authentication

- **Concept**: Register Django REST Framework's authtoken module and run migrations to create the token tables.
- **Code**: Add `'rest_framework.authtoken'` to `INSTALLED_APPS` in `gamekey_platform/settings.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'rest_framework.authtoken', # Enable DRF Token Table
    'games',
]
```

  Run the database migrations:

```
python manage.py migrate
```

- **Verify**: Confirm that the table `authtoken_token` is successfully created in the database (you can check using the Django Admin dashboard or dbshell).
- ⚠ **Common Pitfalls**:
  - **Missing migrations**: If you configure Token Auth but don't run `migrate`, Django will throw a database error (`no such table: authtoken_token`) on the first auth request.

Step 3.2: Configure Global Auth settings

- **Concept**: Update Django settings to enforce Token Authentication and `IsAuthenticatedOrReadOnly` permissions across all endpoints by default.
- **Code**: Add or update `REST_FRAMEWORK` settings in `gamekey_platform/settings.py`:

```
REST_FRAMEWORK = {
    # Default authentication schemes to process incoming requests
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    # Default authorization permissions to evaluate request permissions
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ],
}
```

- **Verify**: Try sending a `POST` request to `http://localhost:8000/api/games/` without an authentication header and verify that the API returns a `401 Unauthorized` status.

Step 3.3: Creating Custom Object-Level Permissions

- **Concept**: We want to make sure that a publisher can only update or delete *their own* games. We will create a custom permission class that checks the owner of the publisher profile.
- **Code**: Create `games/permissions.py`:

```
from rest_framework.permissions import BasePermission, SAFE_METHODS


class IsOwnerOrReadOnly(BasePermission):
    # Override object-level permission check method
    def has_object_permission(self, request, view, obj):
        # Read-only permissions are allowed for any request (GET, HEAD, OPTIONS)
        if request.method in SAFE_METHODS:
            return True

        # Write permissions are only allowed if the game's publisher user matches the logged-i
        return obj.publisher.user == request.user
```

Now apply this custom permission to the `GameViewSet` by updating `games/viewsets.py`:

```
from rest_framework import viewsets
from .models import Game, Publisher
from .serializers import GameSerializer, PublisherSerializer
from .permissions import IsOwnerOrReadOnly # Import custom permission


class GameViewSet(viewsets.ModelViewSet):
    queryset = Game.objects.all()
    serializer_class = GameSerializer
    # Override the global permission list with custom permission class
    permission_classes = [IsOwnerOrReadOnly]


class PublisherViewSet(viewsets.ModelViewSet):
    queryset = Publisher.objects.all()
    serializer_class = PublisherSerializer
```

- **Verify**: Ensure that the custom permission class imports and compiles without errors.
- ⚠ **Common Pitfalls**:
  - ○ **`has_object_permission` not firing on lists**: Note that `has_object_permission` is only called for detail views (`retrieve`, `update`, `destroy`). It is not called for `list` or `create` requests.

Step 3.4: Building the User Registration Endpoint

- **Concept**: Create a view that allows new users to sign up, hashes their passwords, and immediately generates and returns an auth token. Since unregistered users don't have a token, we must explicitly bypass global permissions using `@permission_classes([AllowAny])`.
- **Code**: Add the registration function to `games/views.py`:

```
from django.contrib.auth.models import User
from rest_framework.authtoken.models import Token
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from rest_framework import status


@api_view(['POST'])
# Overwrite global permissions for this route to let anyone access it without credentials
@permission_classes([AllowAny])
def register(request):
    username = request.data.get('username')
    password = request.data.get('password')

    if not username or not password:
        return Response({'error': 'Username and password required.'}, status=status.HTTP_400_B

    # create_user is a built-in helper that automatically hashes the password using PBKDF2
    user = User.objects.create_user(username=username, password=password)

    # Generate a token or fetch the existing token for the new user profile
    token, _ = Token.objects.get_or_create(user=user)

    return Response({'token': token.key}, status=status.HTTP_201_CREATED)
```

Now wire the URL endpoint in `gamekey_platform/urls.py`:

```
from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from games.viewsets import GameViewSet, PublisherViewSet
from games.views import register # Import view function

router = DefaultRouter()
router.register(r'games', GameViewSet)
router.register(r'publishers', PublisherViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
    path('api/register/', register), # Register registration view endpoint
]
```

- **Verify**: Send a `POST` request to `http://localhost:8000/api/register/` with a JSON payload containing `username` and `password`. Verify that it returns `201 Created` along with a hex token.
- ⚠ **Common Pitfalls**:
  - **Using `create()` instead of `create_user()`**: If you use `User.objects.create(...)`, Django stores the password in plaintext, meaning the user can never log in because Django's auth system expects a hashed password. Always use `create_user()`.

- **Wrong Authorization Header Format**: DRF Token Auth expects the header format `Token <token>` (with a space and the word `Token`). Be careful of using the `Bearer <token>` format as it will not match Token authentication.

---

4. Check for Understanding (10 min)

> ***Question***: *"What's the difference between `has_permission` and `has_object_permission`? Give an example where you'd use each."*
> ***Answer***: *`has_permission` checks if the user has access to the endpoint in general (checked at the start of the request, e.g., "Is the user logged in?"). `has_object_permission` is only called during detail views (retrieve/update/delete) to check access to a specific database object (e.g., "Is the logged-in user the owner of this game record?").*

> ***Question***: *"Why do we use `create_user()` instead of `create()` when making User objects?"*
> ***Answer***: *`create_user()` is a helper method on Django's User manager that handles password hashing using a secure algorithm (like PBKDF2). Using `create()` directly would write the password to the database as plaintext, exposing it to database administrators or security breaches, and preventing the user from logging in since Django's auth system expects a hashed password.*

> ***Question***: *"If we didn't add `@permission_classes([AllowAny])` to the register view, what would happen when a new user tries to register?"*
> ***Answer***: *The registration request would be blocked by the global permission default (`IsAuthenticatedOrReadOnly`), returning a `401 Unauthorized` or `403 Forbidden` response. Since a new user does not have a token yet, they would be unable to register.*

> ***Question***: *"What does `IsAuthenticatedOrReadOnly` allow vs deny? Is it the right default for this API?"*
> ***Answer***: *It allows anyone (authenticated or anonymous) to perform safe, read-only HTTP methods (`GET`, `HEAD`, `OPTIONS`), but restricts writing methods (`POST`, `PUT`, `PATCH`, `DELETE`) to authenticated users. It is an excellent default for this API because it keeps the game catalog publicly browsable while securing modifications and purchases.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - `POST /api/register/` returns a JSON response containing an authentication token.
  - `POST /api/games/` without an `Authorization` header returns a `401 Unauthorized` error.

- [ ] `POST /api/games/` with the header `Authorization: Token <token>` successfully creates the game and returns `201 Created`.
- [ ] `PATCH /api/games/1/` with a token belonging to a user *other* than the owner of the game's publisher returns `403 Forbidden`.